

Desafio 1 — Agente de Contas a Pagar: guia completo

Oct 6, 2026 · @Paulo Arthur

1. Visão geral para quem não é técnico

O Agente de Contas a Pagar é um assistente digital do setor financeiro. Ele recebe as notas fiscais e os boletos dos fornecedores, confere tudo sozinho e só chama uma pessoa quando algo está fora do normal.

O problema que ele resolve

Toda empresa compra de fornecedores: aço, embalagens, frete, equipamentos de segurança. Cada compra gera uma nota fiscal e, geralmente, um boleto. Antes de pagar, alguém do financeiro precisa:

1. Abrir o e-mail e baixar a nota e o boleto.
2. Digitar os dados no sistema da empresa (fornecedor, valor, vencimento, itens).
3. Conferir se a nota bate com o que foi comprado (o pedido de compra) e com o que de fato chegou no almoxarifado.
4. Verificar se o boleto é mesmo daquele fornecedor.
5. Organizar os pagamentos por data de vencimento.

Feito à mão, isso leva de 5 a 10 minutos por documento, e os erros custam caro:

- **Pagamento em duplicidade:** o fornecedor reenvia a mesma nota e ela é paga duas vezes.
- **Cobrança indevida:** a nota cobra um preço maior ou uma quantidade maior do que a combinada, e ninguém percebe.
- **Golpe do boleto:** um criminoso envia um boleto com o nome do fornecedor verdadeiro, mas o dinheiro vai para a conta dele.
- **Multas e juros:** um vencimento passa despercebido.
- **Falta de rastreabilidade:** depois, ninguém sabe dizer quem aprovou um pagamento.

O que a aplicação faz

O agente faz o trabalho repetitivo e deixa para a pessoa só as decisões que exigem julgamento. Para cada documento ele dá um de três veredictos:

Veredicto	Significado	O que acontece
APROVADO	Está tudo certo	Vai para a lista de pagamentos, sem ninguém precisar olhar
REVISÃO	Há algo estranho, mas pode ter explicação	Vai para a fila do analista, com o motivo escrito
REJEITADO	Há um problema grave (fraude, duplicidade, documento adulterado)	É bloqueado e não pode ser pago

Uma analogia: é como um conferente experiente que confere 100% das notas em segundos, nunca se cansa e deixa um bilhete explicando cada caso duvidoso.

Para quem é

- **Analista de contas a pagar:** deixa de digitar e conferir, e passa a revisar só as exceções.
- **Gestor financeiro:** enxerga indicadores, valores bloqueados e o que vence nos próximos dias.
- **Controladoria e auditoria:** têm o registro de cada decisão, com autor e horário.

O objetivo de negócio

Reduzir o tempo de entrada de documentos de cerca de 200 horas para cerca de 25 horas por mês (estimativa para 2.000 documentos mensais), zerar pagamentos duplicados e golpes de boleto, e evitar multas por atraso. O detalhamento dessas contas está na seção 11.

O que é "agente" aqui, em uma frase

Além de seguir regras fixas, a aplicação usa inteligência artificial para ler documentos em PDF e para investigar os casos estranhos, como um analista faria: consulta o pedido, olha o histórico do fornecedor e escreve uma justificativa. A seção 4 explica esse e todos os outros termos com calma.

2. A jornada do usuário: todas as funcionalidades

O usuário pode usar a aplicação de três jeitos: pelo painel no navegador, conversando com o Claude ou por comandos no terminal. Os três usam o mesmo motor e o mesmo banco de dados.

2.1 Os documentos chegam

Na demonstração, os documentos ficam numa pasta chamada **inbox** (caixa de entrada). Em produção, um robô copiaria para lá os anexos do e-mail de notas fiscais. A aplicação aceita três tipos:

- **NF-e em XML:** o arquivo oficial da nota fiscal eletrônica, com todos os dados organizados.
- **DANFE em PDF:** a versão "impressa" da nota, usada quando o fornecedor não manda o XML.
- **Boleto em PDF:** o documento de cobrança.

2.2 Processar a caixa de entrada

No painel, o botão **Processar novos da inbox** lê todos os documentos novos. Para cada um, em segundos, a aplicação:

1. Identifica o tipo do documento.
2. Extrai os dados: fornecedor, CNPJ, número, itens, valores, vencimento, pedido de compra.
3. Roda de 10 a 15 verificações (seção 7).
4. Decide: aprovado, revisão ou rejeitado.
5. Grava o resultado, as verificações e um registro de auditoria.

As notas são processadas antes dos boletos. Assim, quando um boleto é analisado, a nota que o justifica já está no sistema.

2.3 Indicadores no topo do painel

Indicador	O que mostra
Documentos	Total processado
Aprovados	Quantos seguiram sem intervenção
Em revisão	Tamanho da fila do analista
Decididos sem intervenção	Percentual decidido pela máquina (regras ou agente), sem humano
Valor bloqueado	Soma em reais dos documentos rejeitados: dinheiro que não saiu indevidamente

2.4 Fila de revisão

A aba **Fila de revisão** mostra só os casos duvidosos. Cada um vem com:

- a justificativa em português ("OL-68: unitário R\$ 420,12 está +8,0% acima do pedido");
- a lista de verificações com ícones:  ok,  alerta,  crítico;
- um campo de observação e os botões **Aprovar** e **Rejeitar**.

Ao decidir, o analista informa o nome. A decisão vai para a trilha de auditoria como humano : <nome> .

2.5 Vencimentos

A aba **Vencimentos** lista os documentos aprovados que vencem nos próximos N dias (o analista escolhe N num controle deslizante), em ordem de data e com o total a pagar. Documentos que vencem em até 3 dias recebem o aviso **PRIORIDADE**.

2.6 Todos os documentos e detalhes

A aba **Todos** mostra a tabela completa: status, arquivo, tipo, fornecedor, valor, vencimento, quem decidiu, como os dados foram extraídos e a justificativa. Em **Detalhar documento**, o analista vê todos os dados extraídos, inclusive item a item.

2.7 Auditoria

A aba **Auditoria** mostra cada evento em ordem: qual documento, o que aconteceu, quem fez e quando. Nada é apagado desse registro.

2.8 Envio manual

Na barra lateral, o analista pode enviar um XML ou PDF e ver a decisão na hora. Se reenviar um arquivo já processado, ele é rejeitado por duplicidade, o que mostra a proteção funcionando.

2.9 Conversando com o agente (via Claude)

Com o servidor MCP registrado no Claude Code ou no Claude Desktop, o analista conversa em linguagem natural:

- "O que está pendente de revisão e por quê?"
- "Aprove a pendência 4, o reajuste foi acordado por e-mail. Meu nome é Paulo."
- "Quanto tenho a pagar nos próximos 7 dias?"
- "Me mostra o histórico do fornecedor de embalagens."

O Claude escolhe a ferramenta certa, chama o agente e responde em português. As 8 ferramentas estão na seção 9.

2.10 Pelo terminal

Para quem prefere linha de comando ou quer agendar tarefas:

Comando	O que faz
<code>python -m ap_agent processar</code>	Processa a inbox (<code>--offline</code> força o modo sem IA)
<code>python -m ap_agent pendencias</code>	Lista a fila de revisão
<code>python -m ap_agent vencimentos --dias 7</code>	Lista o que vence no período
<code>python -m ap_agent exportar</code>	Gera uma planilha Excel com todos os lançamentos

2.11 Receber as notas por e-mail

O agente pode buscar as notas direto numa caixa de e-mail, sem ninguém copiar arquivos para a pasta inbox. O fornecedor manda a nota por e-mail e, ao clicar em **Buscar e-mails** no painel (ou pedir ao Claude "busque as notas no e-mail"), ela aparece decidida.

1. O agente abre a caixa configurada e procura e-mails **não lidos**.
2. Salva na inbox só os anexos `.xml` e `.pdf`, de até 10 MB. Outros anexos (`.exe`, `.zip`, imagens) são ignorados e listados.
3. Marca cada e-mail como lido. Nenhum e-mail é apagado.
4. Processa os anexos como qualquer documento: notas antes de boletos.
5. Registra na auditoria um evento `EMAIL_RECEBIDO`, com remetente, assunto e anexos.

Há três formas de disparar: o comando `python -m ap_agent ler-email`, o botão no painel e a ferramenta MCP `buscar_emails`.

2.12 Testar com notas reais

O script `scripts/preparar_notas_reais.py` monta um ambiente separado (`notas_reais/`, fora do git) a partir das suas próprias NF-e em XML. Ele cadastra o emitente como fornecedor e cria um pedido e um recebimento iguais à nota.

Modo	O que o script faz	Resultado esperado
Padrão (<code>--processar</code>)	Pedido igual à nota	APROVADO
<code>--divergente --processar</code>	Pedido com preço 5% menor	REVISÃO, com o alerta de preço

O teste com uma nota no formato de compra pessoal (sem pedido, destinatário CPF e sem duplicata) deu exatamente esses dois resultados.

3. Os 11 cenários da demonstração

Os 11 documentos da demonstração são fictícios e foram criados para que cada um teste uma situação real do dia a dia. O resultado esperado é 4 aprovados, 3 em revisão e 4 rejeitados.

O cenário de fundo: a empresa "Indústria Exemplo Ltda" tem 5 fornecedores cadastrados e 7 pedidos de compra. Para cada pedido, o almoxarifado registrou o que de fato chegou.

#	Arquivo	Situação	Resultado	Por quê
1	<code>nfe_001_aco_ok.xml</code>	Nota de chapas de aço, tudo certo	APROVADO	Fornecedor cadastrado, chave válida, preço e quantidade batem com pedido e recebimento
2	<code>nfe_002_embalagens_ok.xml</code>	Nota com 2 itens (caixas e fita)	APROVADO	Os dois itens conferem com o pedido PC-1002
3	<code>boleto_002_embalagens.pdf</code>	Boleto da nota 2	APROVADO	Dígitos da linha digitável conferem, valor igual ao da nota, beneficiário é o fornecedor cadastrado
4	<code>nfe_003_lubrificante_preco_acima.xml</code>	Preço 8% acima do pedido	REVISÃO	A tolerância é 2%. Pode ser reajuste combinado, então cabe a um humano decidir

#	Arquivo	Situação	Resultado	Por quê
5	nfe_004_epi_qtd_maior_que_recebida.xml	Cobrou 500 luvas, chegaram 400	REVISÃO	Pagar 500 seria pagar por 100 que não chegaram. Cabe pedir nota complementar ou devolução
6	nfe_005_fornecedor_desconhecido.xml	Emitente fora do cadastro	REJEITADO	Ninguém paga empresa não homologada. Além disso, o pedido citado pertence a outro fornecedor
7	nfe_006_duplicada_do_001.xml	Reenvio da nota 1	REJEITADO	Mesma chave de acesso de um documento já lançado: seria pagamento em dobro
8	nfe_007_chave_invalida.xml	Chave de acesso adulterada	REJEITADO	O dígito verificador não fecha: sinal de documento falso ou corrompido
9	danfe_008_frete_somente_pdf.pdf	Fornecedor só mandou o PDF	APROVADO com IA / REVISÃO sem IA	A IA lê o PDF com alta confiança. Sem IA, a leitura por padrões de texto é marcada como incerta e vai para um humano
10	boleto_009_fraude_beneficiario.pdf	Golpe do boleto	REJEITADO	Usa o nome "Embalagens Sol Nascente S.A.", mas o CNPJ do beneficiário não é o cadastrado
11	nfe_010_paletes_vence_em_2_dias.xml	Nota correta que vence em 2 dias	APROVADO + PRIORIDADE	Está tudo certo; o aviso de prioridade evita multa por atraso

Por que o golpe do boleto é o cenário mais importante

No golpe, o criminoso intercepta ou falsifica o e-mail do fornecedor e manda um boleto idêntico ao verdadeiro. A diferença está só nos dados do beneficiário, quem recebe o dinheiro. Um humano com pressa não percebe.

O agente compara o CNPJ do beneficiário com o cadastro. Se o nome é de um fornecedor conhecido e o CNPJ é outro, ele bloqueia e escreve: "suspeita de boleto adulterado". Um único golpe barrado costuma pagar o projeto inteiro.

Por que existe a "revisão" e não só aprovado/rejeitado

Nem toda divergência é erro. Um preço acima do pedido pode ser um reajuste aprovado por telefone. A máquina não sabe disso, então não decide sozinha: separa o caso, explica e deixa a palavra final com a pessoa.

4. Glossário: os conceitos, da teoria ao projeto

Cada termo tem duas partes: **o que é** (definição geral, válida fora deste projeto) e **no projeto** (como ele aparece aqui, com o arquivo onde está). Os termos estão agrupados por tema.

4.1 Inteligência artificial

Inteligência artificial (IA)

- O que é: área da computação que cria sistemas capazes de tarefas que normalmente exigem inteligência humana, como ler, interpretar, resumir e decidir.
- No projeto: a IA lê PDFs de notas e boletos e investiga casos duvidosos. As regras fixas cuidam do resto.

LLM (Large Language Model, grande modelo de linguagem)

- O que é: tipo de IA treinada com enormes volumes de texto. Recebe um texto (o *prompt*) e gera uma resposta. ChatGPT, Gemini e Claude são LLMs.
- No projeto: usamos o **Claude**, da Anthropic, em dois modelos: `claude-haiku-4-5` (mais barato, para ler PDFs) e `claude-sonnet-5-5` (mais capaz, para investigar exceções e para reler PDFs difíceis).

Prompt e prompt de sistema

- O que é: *prompt* é a instrução dada ao modelo. O *prompt de sistema* é uma instrução fixa que define o papel e as regras do modelo em toda a conversa.

- No projeto: `PROMPT_EXTRACAO` (em `extract/pdf_documento.py`) diz como ler o documento. `SYSTEM` (em `agente.py`) define o agente como "analista de Contas a Pagar" e fixa a política: crítico vira rejeitado, alerta vira no mínimo revisão.

Token

- O que é: a unidade em que o LLM lê e escreve texto, aproximadamente um pedaço de palavra. As APIs cobram por milhão de tokens de entrada e de saída.
- No projeto: um PDF de uma página custa cerca de 3 mil tokens de entrada. O custo estimado é de US\$ 0,006 por PDF no Haiku e de US\$ 0,044 por investigação do agente.

Agente de IA

- O que é: um LLM que, além de responder, **age**. Recebe um objetivo e um conjunto de ferramentas, decide quais usar e em que ordem, lê os resultados e repete até concluir. É diferente de um chatbot, que só conversa.
- No projeto: o agente recebe um documento com problema e investiga. Pode consultar o pedido de compra, o recebimento no almoxarifado e o histórico do fornecedor. No fim, registra a decisão com justificativa e uma ação sugerida (por exemplo, "pedir carta de correção ao fornecedor").

Ferramenta (tool) e tool use

- O que é: uma função do seu programa que o LLM pode pedir para executar. Você descreve a ferramenta (nome, para que serve, parâmetros). O modelo responde "quero chamar `consultar_pedido` com `pedido = PC-1004`". Seu código executa e devolve o resultado ao modelo.
- No projeto: 4 ferramentas em `agente.py`: `consultar_pedido`, `consultar_recebimento`, `historico_fornecedor` e `registrar_decisao` (a que encerra a investigação).

Loop agêntico

- O que é: o ciclo pedir → executar ferramenta → devolver resultado → pedir de novo, que se repete até o agente terminar.
- No projeto: a função `decidir_com_agente` faz no máximo 6 voltas. Se o agente não concluir ou a API falhar, vale a decisão das regras.

Structured output (saída estruturada)

- O que é: recurso que obriga o LLM a responder num formato exato, definido por um esquema (JSON Schema). Assim o programa não precisa "adivinhar" onde está cada informação num texto livre.

- No projeto: a leitura de PDF usa `client.messages.parse(..., output_format=ExtraçãoLLM)`. A resposta já chega como um objeto validado, com campos como `cnpj_emitente`, `valor_total` e `confiança`.

Confiança da extração

- O que é: uma nota de 0 a 1 de quão certo o sistema está sobre os dados extraídos.
- No projeto: o próprio Claude informa a confiança. Abaixo de 0,85, o documento é relido pelo modelo maior. Se continuar baixa, vai para revisão humana. A leitura sem IA recebe sempre 0,6, de propósito.

Guard-rail (trava de segurança)

- O que é: um limite rígido que impede a IA de tomar ações indesejadas, por mais convincente que pareça sua resposta.
- No projeto: a decisão final é sempre a **mais restritiva** entre as regras e o agente. O agente pode ser mais cauteloso, nunca mais permissivo. Se as regras rejeitam e o agente diz "aprovar", vale a rejeição, e o registro anota "agente sugeriu APROVADO".

Fallback (plano B automático)

- O que é: um caminho alternativo usado automaticamente quando o principal falha.
- No projeto: sem chave de API, o PDF é lido por regex e a decisão sai só das regras. Se a API cair no meio, vale a decisão das regras. O processo nunca para por causa da IA.

Human-in-the-loop (humano no circuito)

- O que é: um desenho em que a máquina faz o volume e a pessoa decide os casos ambíguos.
- No projeto: é a fila de revisão, com a decisão humana registrada na auditoria.

4.2 Integração

API (Application Programming Interface)

- O que é: um "balcão de atendimento" para programas. Define pedidos que um sistema aceita de outro e as respostas que devolve, geralmente pela internet.
- No projeto: chamamos a **Claude API** da Anthropic pela biblioteca oficial `anthropic` (Python). Em produção, o agente chamaria também a API do ERP e da SEFAZ.

SDK (Software Development Kit)

- O que é: biblioteca pronta que facilita usar uma API numa linguagem de programação.
- No projeto: o SDK `anthropic` para Python e o SDK `mcp` para o servidor MCP.

MCP (Model Context Protocol)

- O que é: um padrão aberto, criado pela Anthropic, para conectar assistentes de IA a sistemas externos. Funciona como uma "tomada universal": um sistema que expõe um **servidor MCP** pode ser usado por qualquer **cliente MCP** (Claude Code, Claude Desktop, n8n e outros), sem integração específica. O servidor oferece *ferramentas* (ações), *recursos* (dados para leitura) e *prompts*.
- No projeto: `mcp_server.py` transforma o agente num servidor MCP com 8 ferramentas e 1 recurso (`contas-a-pagar://kpis`). Funciona de dois jeitos: **stdio** (o Claude Code abre o programa e conversa com ele pela entrada e saída padrão) e **HTTP** (`--http` , em `http://127.0.0.1:8000/mcp` , para o n8n ou outros clientes de rede).

Tool use × MCP: qual a diferença?

- O tool use do `agente.py` é interno: as ferramentas existem só dentro daquela investigação. O MCP é externo: expõe o sistema inteiro para outro assistente usar. No primeiro, nosso código controla o Claude; no segundo, o Claude do usuário controla nosso sistema.

ERP (Enterprise Resource Planning)

- O que é: o sistema de gestão central da empresa (SAP, TOTVS, Omie), onde ficam cadastros, pedidos, estoque e financeiro.
- No projeto: na demonstração, os dados do ERP são simulados por arquivos CSV (`data/fornecedores.csv` , `pedidos.csv` , `recebimentos.csv`). O módulo `cadastros.py` isola esse acesso: em produção, só ele muda.

RPA (Robotic Process Automation)

- O que é: automação de tarefas repetitivas que uma pessoa faria no computador (copiar, colar, digitar, conferir).
- No projeto: o agente é uma automação de RPA "inteligente". Além de repetir passos, interpreta documentos e explica decisões.

IMAP e senha de app

- O que é: IMAP é o protocolo padrão para um programa ler uma caixa de e-mail no servidor, sem apagar as mensagens. A *senha de app* é uma senha específica para um programa, gerada pelo Gmail ou Outlook, que não dá acesso à conta toda e pode ser revogada a qualquer momento. Exige verificação em duas etapas.
- No projeto: `ingest_email.py` usa a biblioteca padrão `imaplib` para buscar e-mails não lidos em `imap.gmail.com` com uma senha de app guardada no `.env` .

Provedor de IA e API compatível com OpenAI

- O que é: vários serviços de IA (OpenRouter, Ollama, Groq e outros) aceitam pedidos no mesmo formato da API da OpenAI. Um programa escrito para esse formato troca de modelo só mudando o endereço e a chave.

- No projeto: `llm.py` tem dois provedores com a mesma interface: Claude (padrão) e "OpenAI-compatível". A escolha é feita no `.env` por `AP_LLM_PROVEDOR`.

OpenRouter e Qwen

- O que é: o OpenRouter é um serviço que dá acesso, com uma única chave, a centenas de modelos de várias empresas, alguns gratuitos com limite diário. Qwen é uma família de modelos de linguagem de código aberto da Alibaba.
- No projeto: é a opção gratuita para ligar a IA sem a chave do Claude. No plano gratuito, os dados enviados podem ser usados pelo provedor; por isso, só notas fictícias ou suas.

4.3 Documentos fiscais brasileiros

NF-e (Nota Fiscal Eletrônica) e XML

- O que é: a nota fiscal oficial no Brasil é um arquivo **XML** assinado digitalmente e autorizado pela SEFAZ. XML é um formato de texto com etiquetas (`<emit><CNPJ>...` `</CNPJ></emit>`) que organiza os dados de forma que um programa lê com 100% de precisão. O layout atual é a versão 4.00.
- No projeto: `extract/nfe_xml.py` lê as etiquetas oficiais: `infNFe` (a nota), `emit` (emitente), `dest` (destinatário), `det/prod` (itens), `xPed` (número do pedido do comprador), `ICMSTot` (totais) e `cobr/dup` (vencimentos).

DANFE

- O que é: o Documento Auxiliar da NF-e, a representação em PDF ou papel da nota. Serve para leitura humana, não é a nota em si.
- No projeto: quando só há DANFE, a IA lê o PDF. O teste usa o arquivo `danfe_008`.

Chave de acesso

- O que é: o "RG" de 44 dígitos de cada NF-e. Reúne UF, ano e mês, CNPJ do emitente, modelo, série, número, tipo de emissão, um código e, por último, um dígito verificador.
- No projeto: a chave é usada para validar a nota e para detectar duplicidade.

Dígito verificador (DV), módulo 10 e módulo 11

- O que é: um dígito calculado a partir dos outros por uma fórmula. Se alguém erra ou altera um número, a conta não fecha. "Módulo 11" e "módulo 10" são as fórmulas: multiplicam cada dígito por um peso, somam e usam o resto da divisão por 11 ou por 10.
- No projeto: `brutils.py` implementa o DV do CNPJ, o DV da chave de acesso (módulo 11) e os DVs do boleto (módulo 10 em cada campo e módulo 11 no geral). É assim que a nota 7, adulterada, é pega.

CNPJ

- O que é: o cadastro nacional de cada empresa: 14 dígitos, sendo os 2 últimos verificadores.
- No projeto: o CNPJ é validado matematicamente, comparado com o cadastro de fornecedores e, no boleto, usado para pegar o golpe.

Boleto, linha digitável e código de barras

- O que é: o boleto segue o padrão FEBRABAN. O código de barras tem 44 dígitos: banco, moeda, DV geral, fator de vencimento, valor e um campo livre do banco. A **linha digitável** (47 dígitos) é o mesmo conteúdo reorganizado em 5 campos, com DVs extras para quem digita.
- No projeto: `decodificar_linha_digitavel` confere todos os DVs e extrai o valor e o vencimento. Se o valor impresso for diferente do valor codificado, o boleto é rejeitado.

Fator de vencimento

- O que é: número de 4 dígitos no boleto que representa a data, contando dias a partir de uma data-base. Como 4 dígitos acabariam, a FEBRABAN reiniciou a contagem em 22/02/2025 (fator 1000).
- No projeto: `brutils.py` usa a nova data-base de 22/02/2025.

3-way match (conferência em três vias)

- O que é: prática de controle interno que só libera o pagamento quando três documentos concordam: o **pedido de compra** (o que foi pedido e a que preço), o **recebimento** (o que chegou) e a **nota fiscal** (o que está sendo cobrado).
- No projeto: `validar_match_pedido` compara item a item. Preço acima da tolerância de 2%, quantidade acima do pedido ou acima do recebido geram alerta.

SEFAZ

- O que é: a Secretaria da Fazenda estadual, que autoriza as NF-e e permite consultar se uma nota está válida ou cancelada.
- No projeto: a consulta à SEFAZ está prevista no plano de implantação. Não está na demonstração.

4.4 Programação e dados

Python

- O que é: linguagem de programação popular em automação, dados e IA.
- No projeto: toda a aplicação é escrita em Python 3.12.

Regex (expressão regular)

- O que é: uma "linguagem de busca de padrões" em texto. Por exemplo, `\d{2}/\d{2}/\d{4}` encontra qualquer data no formato 05/10/2026. É rápida e gratuita, mas frágil: se o layout do documento muda, o padrão para de encontrar.
- No projeto: é o plano B da leitura de PDF (`_extrair_por_regex`). O texto é extraído do PDF com `pypdf` e buscado com padrões como `CNPJ:\s*([\d./-]{18})` e `Vencimento:\s*(\d{2}/\d{2}/\d{4})` . Por ser frágil, recebe confiança 0,6 e sempre passa por revisão, no caso de DANFE.

Parser (analisador)

- O que é: código que lê um formato estruturado e transforma em dados utilizáveis.
- No projeto: o parser de XML da NF-e usa `xml.etree.ElementTree` , da biblioteca padrão do Python.

Determinístico × probabilístico

- O que é: um processo determinístico dá sempre a mesma saída para a mesma entrada (uma conta, uma regra). Um LLM é probabilístico: pode variar e pode errar.
- No projeto: esse é o princípio central. Tudo o que pode ser determinístico (ler XML, calcular DV, conferir somas) não usa IA. A IA fica só onde é preciso interpretar.

JSON

- O que é: formato de texto leve para trocar dados entre programas, com pares chave e valor: `{"valor": 2850.0}` .
- No projeto: é o formato das respostas da IA, das ferramentas e do campo `dados_json` do banco.

Pydantic

- O que é: biblioteca Python que define "moldes" de dados com tipos e valida automaticamente se os dados se encaixam.
- No projeto: `models.py` define `DocumentoFiscal` , `Item` , `Verificacao` , `Decisao` e `ExtracaoLLM` . Este último vira o esquema que o Claude é obrigado a seguir.

Enum (enumeração)

- O que é: um tipo com um conjunto fechado de valores possíveis.
- No projeto: `Status` (APROVADO, REVISAO, REJEITADO, PAGO), `Severidade` (OK, ALERTA, CRITICO) e `TipoDocumento` (NFE_XML, DANFE_PDF, BOLETO).

Banco de dados e SQLite

- O que é: um banco de dados guarda informações de forma organizada e consultável. SQLite é um banco que cabe num único arquivo, sem servidor; é ótimo para protótipos.

- No projeto: `data/contas_a_pagar.db`, com 3 tabelas: `documentos`, `verificacoes` e `auditoria`. Em produção, a troca seria por PostgreSQL.

SQL

- O que é: a linguagem para consultar e alterar bancos de dados (`SELECT`, `INSERT`, `UPDATE`).
- No projeto: `store.py` concentra todo o SQL.

Hash (SHA-256)

- O que é: uma "impressão digital" de um arquivo: um código calculado a partir do conteúdo. Arquivos idênticos têm o mesmo hash; mudou um byte, muda o hash.
- No projeto: cada arquivo recebe seu hash. Reenviar o mesmo arquivo é detectado como duplicidade, mesmo com outro nome.

Trilha de auditoria (append-only)

- O que é: um registro cronológico de quem fez o quê e quando, onde só se acrescenta e nunca se apaga.
- No projeto: a tabela `auditoria` recebe um evento a cada decisão, seja da regra, do agente ou de um humano.

CSV

- O que é: uma tabela em texto simples, com colunas separadas por vírgula; abre no Excel.
- No projeto: simula os cadastros do ERP e lista os cenários esperados.

4.5 Interfaces e qualidade

CLI (Command Line Interface)

- O que é: programa usado por comandos digitados no terminal.
- No projeto: `python -m ap_agent processar | pendencias | vencimentos | exportar` (arquivo `__main__.py`).

Streamlit

- O que é: biblioteca Python que cria painéis web interativos só com código Python, sem HTML ou JavaScript.
- No projeto: `dashboard.py`, o painel do analista em `http://localhost:8501`.

stdio e HTTP (transportes)

- O que é: duas formas de dois programas conversarem. *stdio* é a entrada e saída padrão de um processo, no mesmo computador. *HTTP* é o protocolo da web, pela rede.

- No projeto: o servidor MCP fala stdio com o Claude Code e HTTP com o n8n.

Testes automatizados e pytest

- O que é: código que verifica se outro código funciona, rodado com um comando. `pytest` é a ferramenta padrão de testes em Python.
- No projeto: 10 testes em `tests/test_agente.py` (seção 10).

Mock (simulação)

- O que é: um objeto falso que imita um serviço real num teste, para testar sem depender dele.
- No projeto: um cliente Claude falso responde com um roteiro pré-definido, para testar o loop do agente sem gastar API.

Variáveis de ambiente e `.env`

- O que é: configurações passadas ao programa de fora do código, como chaves e parâmetros. O arquivo `.env` as guarda localmente e não vai para o repositório.
- No projeto: `ANTHROPIC_API_KEY`, modelos, CNPJ da empresa, tolerância de preço e dias de alerta (`config.py`, modelo em `.env.example`).

Git

- O que é: sistema de controle de versões: guarda o histórico de cada mudança no código.
- No projeto: o repositório `Case-GoGroup` com o commit inicial. Dados, banco e `.env` ficam fora (`.gitignore`).

4.6 Negócio e governança

Straight-through processing (STP)

- O que é: percentual de transações processadas do início ao fim sem toque humano.
- No projeto: é o KPI "decididos sem intervenção", com meta de pelo menos 70% em produção.

Shadow mode (modo sombra)

- O que é: fase em que o sistema novo roda em paralelo ao processo atual, só sugerindo, para comparar antes de ligar de verdade.
- No projeto: é a fase 2 do plano de implantação, com 4 semanas e meta de zero falso-aprovado.

LGPD

- O que é: a Lei Geral de Proteção de Dados, que regula o uso de dados pessoais no Brasil.

- No projeto: o plano prevê mandar à IA só os PDFs sem XML, reter o mínimo, mascarar dados de pessoas físicas e ter contrato de tratamento de dados com o provedor.

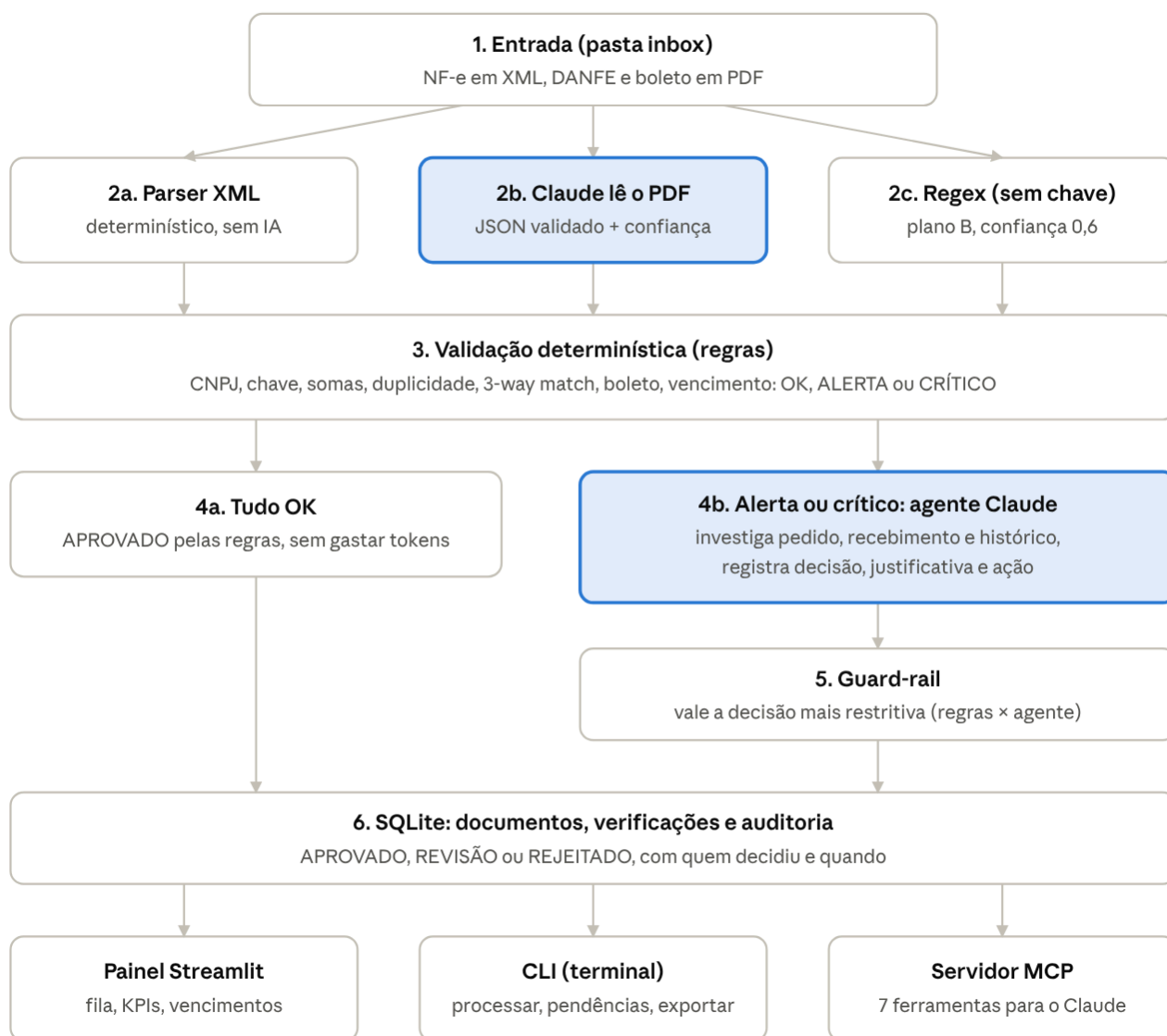
KPI e ROI

- O que é: KPI é um indicador-chave de desempenho. ROI é o retorno sobre o investimento; *payback* é o tempo para o projeto se pagar.
- No projeto: os KPIs e o ROI estão no plano de implantação (seção 11).

5. Arquitetura técnica

A aplicação é uma linha de montagem de seis etapas. Cada documento entra, é lido, conferido e decidido, e o resultado fica disponível em três interfaces que compartilham o mesmo banco.

Cada documento: extração, validação e decisão



fluxo de um documento · 6 etapas, 3 interfaces

As caixas em destaque são as únicas etapas que usam IA. Todo o resto é código determinístico, que dá sempre o mesmo resultado.

5.1 As etapas

1. **Entrada:** `pipeline.py` lista os `.xml` e `.pdf` da inbox.
2. **Extração:** transforma cada arquivo num objeto `DocumentoFiscal` padronizado. O caminho depende do formato:
 - XML vai para o parser determinístico (`extract/nfe_xml.py`);
 - PDF vai para o Claude (`extract/pdf_documento.py`), ou para regex quando não há chave.
3. **Validação:** `validacao.py` roda as regras e devolve uma lista de verificações, cada uma com severidade OK, ALERTA ou CRÍTICO.
4. **Decisão:** `agente.py` converte as verificações numa decisão:
 - sem problemas, aprova direto, sem chamar a IA;
 - com problemas e com chave de API, chama o agente investigador.
5. **Guard-rail:** compara as duas decisões e fica com a mais restritiva.
6. **Persistência:** `store.py` grava documento, verificações e evento de auditoria no SQLite.

5.2 As decisões de projeto e por quê

Decisão	Motivo
XML lido sem IA	O XML é estruturado e oficial. Usar IA só traria custo e risco de erro, sem ganho
IA só nas exceções	Cerca de 75% dos documentos são limpos. Não há por que pagar tokens para aprová-los
Regras como piso de segurança	LLMs podem errar. Regras matemáticas (DV, somas) não. A IA soma, não substitui
Modo offline completo	A demonstração roda em qualquer máquina, e a operação não para se a API cair
Notas antes de boletos	O boleto só é aceito se existir a nota que o justifica (o "lastro")
Cadastros isolados em <code>cadastros.py</code>	Trocar CSV por ERP mexe num arquivo só
Uma base, três interfaces	Painel, terminal e MCP mostram sempre os mesmos dados

Decisão	Motivo
Auditoria só de inserção	Exigência de controle interno: nada some do histórico

5.3 Estrutura de pastas

```
desafio1-agente-nf/
├─ ap_agent/                pacote principal
│   ├── config.py           configuração (.env)
│   ├── models.py           moldes de dados (Pydantic)
│   ├── brutils.py          CNPJ, chave de acesso, boleto
│   ├── extract/
│   │   ├── nfe_xml.py      leitura de XML
│   │   └─ pdf_documento.py leitura de PDF (Claude ou regex)
│   ├── cadastros.py        fornecedores, pedidos, recebimentos
│   ├── validacao.py        regras de negócio
│   ├── agente.py           decisão: regras + agente + guard-rail
│   ├── store.py            banco SQLite
│   ├── pipeline.py         orquestração
│   └─ __main__.py          terminal (CLI)
├─ mcp_server.py            servidor MCP
├─ dashboard.py             painel Streamlit
├─ scripts/gerar_dados_ficticios.py
├─ tests/test_agente.py
├─ data/                   cadastros (CSV) e banco
├─ inbox/                  documentos a processar
└─ docs/                   arquitetura e plano de implantação
```

5.4 Fluxo de execução: o que é feito por IA e o que não é

A regra de projeto é simples: **a IA lê e explica; quem decide são as regras**. A IA entra em só dois pontos (ler PDF e investigar exceções), e nenhum deles pode tornar uma decisão mais permissiva do que as regras permitem. Sem IA, o fluxo inteiro continua funcionando.

#	Etapa	Quem executa	Usa IA?	O que acontece
1	Entrada	<code>ingest_email.py</code> , upload do painel ou pasta <code>inbox</code>	Não	Anexos <code>.xml</code> e <code>.pdf</code> chegam à inbox; outros tipos são ignorados; o e-mail fica marcado como lido e auditado (<code>EMAIL_RECEBIDO</code>)

#	Etapa	Quem executa	Usa IA?	O que acontece
2	Ordenação e duplicidade	<code>pipeline.py</code>	Não	Notas antes de boletos (o boleto precisa achar a nota); arquivo já lançado é pulado
3a	Leitura do XML	<code>nfe_xml.py</code>	Não	Parser determinístico do layout oficial da NF-e; confiança 1,0. NFS-e é reconhecida e recusada com mensagem clara
3b	Leitura do PDF (DANFE ou boleto)	<code>pdf_documento.py</code> + <code>llm.py</code>	Sim	A cadeia de IA devolve um JSON validado pelo Pydantic, com confiança. Se toda a cadeia falhar: regex sobre o texto, com confiança 0,6 (vai para revisão)
4	Validação	<code>validacao.py</code>	Não	21 regras: CNPJ, chave (módulo 11), boleto (FEBRABAN), cadastro, pedido (inclusive inferido), 3-way match, totais, vencimento, duplicidade, golpe do boleto
5	Decisão base	<code>agente.py</code>	Não	Algum CRÍTICO → REJEITADO; algum ALERTA ou confiança < 0,85 → REVISÃO; tudo OK → APROVADO

#	Etapa	Quem executa	Usa IA?	O que acontece
6	Investigação das exceções	<code>agente.py</code> + <code>llm.py</code>	Sim , só se a etapa 5 não deu APROVADO	O agente consulta pedido, recebimento, fornecedor e histórico (4 ferramentas) e escreve justificativa e ação sugerida
7	Guard-rail	<code>agente.py</code>	Não	O status final é o mais restritivo entre regras e agente: a IA pode endurecer, nunca afrouxar
8	Gravação e auditoria	<code>store.py</code>	Não	SQLite: documento, verificações, decisão, quem decidiu (<code>regras</code> , <code>agente</code> : <code><modelo></code> ou <code>humano</code> : <code><nome></code>)
9	Interfaces	<code>dashboard.py</code> , <code>__main__.py</code> , <code>mcp_server.py</code>	Painel e terminal: não. MCP: a IA é o cliente	No MCP, o Claude do usuário entende o pedido e escolhe a ferramenta; o servidor só executa código determinístico

Quantas chamadas de IA na demonstração? São 11 documentos:

- 2 leituras de PDF (a DANFE e o boleto);
- 6 investigações (as 2 revisões e os 4 rejeitados);
- cerca de 8 a 20 chamadas no total, porque cada investigação pode usar algumas voltas de ferramentas.

Os documentos limpos lidos por XML não chamam IA nenhuma.

Por que dividir assim?

- **Regras para o que tem resposta exata.** Dígito verificador, soma e cadastro têm resposta certa, e código erra menos e custa zero.
- **IA para o que é linguístico ou visual.** Ela lê layouts de PDF variados e explica uma divergência em português para o analista.
- **Segurança.** Mesmo uma IA enganada por um PDF malicioso não consegue aprovar um documento que as regras rejeitaram.

6. O código, módulo a módulo

O pacote `ap_agent` tem 13 módulos, cada um com uma responsabilidade. Abaixo, o que cada um faz e as funções principais, na ordem em que um documento passa por eles.

6.1 `config.py` : configuração

Lê o arquivo `.env` com `python-dotenv` e expõe as configurações como constantes:

Configuração	Padrão	Para quê
<code>ANTHROPIC_API_KEY</code>	vazio	Sem ela, roda em modo offline
<code>AP_MODELO_EXTRACAO</code>	<code>claude-haiku-4-5</code>	Modelo que lê PDFs
<code>AP_MODELO_AGENTE</code>	<code>claude-sonnet-5-5</code>	Modelo que investiga e relê PDFs difíceis
<code>AP_CNPJ_EMPRESA</code>	<code>11222333000181</code>	CNPJ da "nossa" empresa (destinatária)
<code>AP_TOLERANCIA_PRECO</code>	2,0%	Variação de preço aceita sem alerta
<code>AP_DIAS_ALERTA_VENCIMENTO</code>	3	Antecedência do aviso de prioridade
<code>AP_CONFIANCA_MINIMA</code>	0,85	Abaixo disso, relê ou manda para revisão

A função `llm_disponivel()` responde se há chave, e é ela que liga ou desliga a IA em todo o sistema.

6.2 `models.py` : os moldes de dados

Define, com Pydantic, a "forma" de cada informação que circula:

- `DocumentoFiscal` : o documento normalizado, igual para XML, DANFE e boleto. Campos: tipo, arquivo, chave de acesso, número, série, emissão, CNPJ e nome do emitente, CNPJ do destinatário, pedido, itens, valores, vencimento, linha digitável, confiança e método de extração.
- `Item` : código, descrição, quantidade, valor unitário e total.
- `ExtracaoLLM` : o esquema que o Claude preenche ao ler um PDF. Cada campo tem uma descrição que orienta o modelo ("somente dígitos", "AAAA-MM-DD").
- `Verificacao` : regra, severidade e mensagem.

- `Decisao` : status, justificativa e quem decidiu (`regras` , `agente:<modelo>` ou `humano:<nome>`).

Padronizar tudo num só `DocumentoFiscal` é o que permite que as regras não precisem saber de onde o dado veio.

6.3 `brutils.py` : matemática dos documentos brasileiros

Funções puras, sem dependências, que implementam os padrões oficiais:

- `cnpj_valido` , `gerar_cnpj` , `formatar_cnpj` : cálculo dos dois dígitos verificadores do CNPJ.
- `chave_acesso_valida` , `gerar_chave_acesso` : módulo 11 sobre os 43 primeiros dígitos da chave.
- `gerar_linha_digitavel` , `decodificar_linha_digitavel` : monta e desmonta o boleto FEBRABAN. A decodificação:
 1. confere os três DVs de campo (módulo 10);
 2. reconstrói o código de barras de 44 dígitos;
 3. confere o DV geral (módulo 11);
 4. extrai o valor (últimos 10 dígitos, em centavos) e o vencimento (fator + data-base de 22/02/2025).

As funções `gerar_*` existem para o gerador de dados fictícios criar documentos válidos, e documentos inválidos de propósito.

6.4 `extract/nfe_xml.py` : leitura de XML

`extrair_nfe_xml(caminho)` abre o XML com `ElementTree` , usando o *namespace* oficial `http://www.portalfiscal.inf.br/nfe` , e copia cada etiqueta para o campo correspondente. Por exemplo, `emit/CNPJ` vira `cnpj_emitente` , `det/prod/qCom` vira a quantidade do item e `cobr/dup/dVenc` vira o vencimento. O pedido de compra vem da etiqueta `xPed` de cada item. A confiança é sempre 1,0, porque o XML é a fonte oficial.

6.5 `extract/pdf_documento.py` : leitura de PDF

`extrair_pdf(caminho)` escolhe o caminho:

- **Com chave:** `_extrair_com_claude` codifica o PDF em base64, envia como bloco `document` junto com o `PROMPT_EXTRACAO` e recebe um `ExtracaoLLM` já validado. Se a confiança vier abaixo de 0,85, repete com o modelo maior.
- **Sem chave:** `_extrair_por_regex` extrai o texto com `pypdf` e aplica padrões: CNPJs, valores "R\$ 1.234,56", datas, linha digitável e itens no formato `CÓDIGO DESCRIÇÃO QTD V.UNIT V.TOTAL` . Distingue boleto de DANFE pela presença de "Beneficiário".

Nos dois casos, o resultado é convertido em `DocumentoFiscal` : datas viram objetos de data e CNPJs ficam só com dígitos. Se o boleto não trouxer o vencimento impresso, ele é calculado a partir da linha digitável.

6.6 `cadastros.py` : dados mestres

Lê os três CSVs e devolve dicionários fáceis de consultar:

- `fornecedores()` : CNPJ → razão social e categoria.
- `pedidos()` : número do pedido → fornecedor e itens (quantidade e preço).
- `recebimentos()` : pedido → quantidade recebida por item.

Usa `@lru_cache` para ler cada arquivo uma vez só. Em produção, estas três funções passariam a consultar o ERP, sem mudar o resto do sistema.

6.7 `validacao.py` : as regras

`validar_documento` é o ponto de entrada e chama, conforme o tipo:

- `validar_cadastrais` : CNPJ, fornecedor e destinatário (todos os tipos);
- a busca de duplicidade em `store.buscar_duplicado` (todos os tipos);
- `validar_nota` : chave, somas, cálculo por item e confiança (notas);
- `validar_match_pedido` : o 3-way match (notas);
- `validar_boleto` : fraude de beneficiário, DVs, valor e lastro (boletos);
- `validar_vencimento` : vencido, sem data ou próximo (todos os tipos).

O catálogo completo está na seção 7.

6.8 `agente.py` : a decisão

Três peças:

1. `decidir_por_regras` : qualquer CRÍTICO vira REJEITADO; senão, qualquer ALERTA vira REVISÃO; senão, APROVADO. A justificativa junta as mensagens.
2. `decidir_com_agente` : o loop agêntico (seção 8).
3. `decidir` : orquestra. Se não há IA ou se as regras aprovaram, devolve a decisão das regras. Senão, chama o agente e aplica o guard-rail com uma tabela de rigor: APROVADO = 0, REVISÃO = 1, REJEITADO = 2. Se o rigor do agente for menor que o das regras, prevalecem as regras e o parecer do agente fica anexado.

6.9 `store.py` : o banco de dados

- `conexao()` : abre o SQLite e cria as tabelas na primeira vez. Grava ao fechar, sem precisar de `commit` manual.

- `buscar_duplicado` : procura um documento não rejeitado com o mesmo hash de arquivo, a mesma chave de acesso ou o mesmo emitente e número.
- `notas_do_fornecedor` : usada para achar o lastro do boleto e para o histórico.
- `salvar` : grava documento, verificações e evento de auditoria.
- `atualizar_status` : muda o status (decisão humana) e audita.
- `listar` , `verificacoes_de` : consultas.

Duplicidade só conta contra documentos **não rejeitados**. Assim, se um fornecedor novo foi rejeitado e depois cadastrado, a nota pode ser reprocessada.

6.10 `pipeline.py` : a orquestração

- `processar_documento(caminho)` : extrai, valida, decide, salva e devolve um resumo, incluindo o tempo gasto em segundos.
- `processar_pasta()` : extrai todos os arquivos primeiro, ordena para deixar boletos por último e só então valida. Um arquivo corrompido vira status ERRO sem derrubar o lote.

6.11 `__main__.py` : o terminal

Usa `argparse` para os subcomandos `processar` , `pendencias` , `vencimentos` e `exportar` . Imprime os status coloridos (verde, amarelo, vermelho) e um resumo final. A exportação usa `pandas` para gerar o Excel.

6.12 Fora do pacote

- `mcp_server.py` : o servidor MCP (seção 9).
- `dashboard.py` : o painel Streamlit (seção 9).
- `scripts/gerar_dados_ficticios.py` : cria fornecedores, pedidos e recebimentos em CSV, além dos 11 documentos da inbox. Os XMLs seguem o layout 4.00; DANFE e boletos são PDFs gerados com `reportlab` . Todas as datas são relativas ao dia em que o script roda, para os vencimentos fazerem sentido em qualquer data.

6.13 `llm.py` : provedores de IA

Concentra toda conversa com modelos de IA, com a mesma interface para os dois provedores: `extrair_pdf(caminho)` e `rodar_agente(system, tools, pedido, executar)` . O resto do sistema chama `provedor()` e não sabe qual modelo está do outro lado.

	ProvedorAnthropic (Claude)	ProvedorOpenAICompat (OpenRouter, Ollama...)
PDF	Enviado inteiro (bloco <code>document</code>)	Texto extraído com <code>pypdf</code> e enviado ao modelo
Formato da resposta	<code>messages.parse</code> com o schema Pydantic	<code>response_format</code> JSON Schema; se o modelo não aceitar, <code>json_object</code> ; validação com Pydantic
Ferramentas do agente	<code>tools</code> no formato Anthropic	As mesmas 4, convertidas para <code>tools / tool_calls</code> (padrão OpenAI)
Erro	<code>anthropic.APIError</code>	<code>openai.OpenAIError</code>

Nos dois casos, qualquer falha devolve o controle ao modo offline: regex na extração, regras na decisão.

6.14 `ingest_email.py` : leitura de e-mail

`buscar_anexos_email()` conecta por IMAP com SSL, busca os e-mails `UNSEEN`, lê cada um sem marcá-lo (`BODY.PEEK`), percorre as partes da mensagem e salva os anexos aceitos. Detalhes de segurança:

- `nome_seguro()` mantém só o nome do arquivo e troca caracteres especiais. Um anexo chamado `../../boleto.pdf` vira `boleto.pdf` dentro da inbox.
- O nome salvo leva o identificador do e-mail (`email7_nota.xml`), para nunca sobrescrever outro arquivo.
- Só depois de salvar, o e-mail é marcado como lido (`\Seen`).

`pipeline.processar_emails()` junta a busca e o processamento, e é usada pelo terminal, pelo painel e pelo MCP.

7. Catálogo de regras de validação

São 21 regras em `validacao.py`, cada uma com uma severidade. **CRÍTICO** bloqueia o pagamento, **ALERTA** manda para revisão humana e **OK** confirma que a checagem passou.

7.1 As três severidades

Severidade	Quando usar	Efeito na decisão
CRÍTICO	Erro objetivo ou risco de fraude: não há explicação legítima	REJEITADO, sem exceção
ALERTA	Divergência que pode ter explicação (reajuste, entrega parcial)	No mínimo REVISÃO
OK	Checagem passou (ou só informa prioridade)	Não bloqueia

7.2 As regras

Regra	Aplica a	Se falhar	O que verifica
cnpj_emitente	Todos	CRÍTICO	Dígitos verificadores do CNPJ do emitente
fornecedor_cadastrado	Todos	CRÍTICO	Se o CNPJ está no cadastro de fornecedores
destinatario	Todos	CRÍTICO	Se o documento é endereçado ao CNPJ da nossa empresa
duplicidade	Todos	CRÍTICO	Mesma chave, mesmo emitente + número ou mesmo arquivo já lançado
chave_acesso	NF-e / DANFE	CRÍTICO	Dígito verificador da chave de 44 dígitos
soma_itens	NF-e / DANFE	CRÍTICO	Soma dos itens igual ao total de produtos (tolerância de R\$ 0,01)
calculado_item	NF-e / DANFE	ALERTA	Quantidade × preço unitário = total do item
confianca_extracao	PDF	ALERTA	Confiança da leitura abaixo de 85%
pedido	NF-e / DANFE	ALERTA ou CRÍTICO	Nota sem pedido (alerta) ou pedido inexistente (crítico)
pedido_fornecedor	NF-e / DANFE	CRÍTICO	Se o pedido pertence ao mesmo fornecedor da nota

Regra	Aplica a	Se falhar	O que verifica
item_no_pedido	NF-e / DANFE	ALERTA	Item da nota que não existe no pedido
preco	NF-e / DANFE	ALERTA	Preço unitário acima do pedido em mais de 2%
quantidade_pedido	NF-e / DANFE	ALERTA	Quantidade faturada acima da pedida
recebimento	NF-e / DANFE	ALERTA	Quantidade faturada acima da recebida no almoxarifado
fraude_beneficiario	Boleto	CRÍTICO	Nome de fornecedor cadastrado com CNPJ diferente do dele
linha_digitavel	Boleto	CRÍTICO	Presença e DVs (módulo 10 e 11) da linha digitável
valor_codigo_barras	Boleto	CRÍTICO	Valor codificado igual ao valor impresso
boleto_x_nota	Boleto	ALERTA ou CRÍTICO	Se existe nota do mesmo beneficiário com o mesmo valor (lastro); crítico se o beneficiário nem é fornecedor
vencimento / vencimento_proximo	Todos	ALERTA / OK	Vencido ou sem data (alerta); vence em até 3 dias (OK com aviso de PRIORIDADE)

Quando todas as checagens do 3-way match passam, aparece a verificação positiva

3way_match : "Nota confere com pedido e recebimento".

7.3 Detalhes que merecem atenção

- **Preço abaixo do pedido não gera alerta.** Só a cobrança a maior é risco para a empresa.
- **Vencimento próximo não bloqueia.** Bloquear uma nota correta que vence amanhã causaria justamente a multa que se quer evitar. Ela é aprovada com o aviso de prioridade.
- **O boleto não recebe alerta de confiança baixa.** A linha digitável tem dígitos verificadores próprios e um valor codificado. Se a leitura errasse um dígito, a conta não fecharia. O boleto, portanto, se autoverifica.

- **Tolerâncias são configuráveis.** A tolerância de preço (2%), os dias de alerta (3) e a confiança mínima (85%) ficam no `.env`. O gestor financeiro ajusta sem mexer no código.

7.4 Duas regras para notas do mundo real

Regra	Severidade	Quando atua	O que faz
<code>pedido_inferido</code>	OK (informativo)	A nota não traz o número do pedido (<code>xPed</code>), o que é comum em notas reais	Procura o único pedido do mesmo fornecedor que contenha todos os itens da nota. Achou um: usa esse pedido e segue o 3-way match. Achou nenhum ou mais de um: mantém o alerta "sem referência a pedido"
<code>vencimento_calculado</code>	OK (informativo)	A nota vem sem duplicata (sem data de vencimento)	Se o fornecedor tem <code>prazo_pagamento_dias</code> no cadastro, o vencimento vira emissão + prazo. Sem prazo cadastrado, continua o alerta "sem data de vencimento"

As duas reproduzem o que um analista faria: localizar o pedido pelo fornecedor e pelos itens, e aplicar a condição de pagamento combinada com o fornecedor.

8. Como a IA é usada

A IA entra em dois pontos: ler PDFs e investigar exceções. Nos dois, ela é cercada por validações determinísticas e nunca tem a palavra final sozinha.

8.1 Leitura de PDF (extração)

O pedido ao Claude. Uma mensagem com dois blocos: o PDF, em base64, como bloco `document`, e o texto do `PROMPT_EXTRACAO`. O prompt pede para:

- extrair os dados exatamente como aparecem;
- usar só dígitos em CNPJ, chave e linha digitável;
- usar datas AAAA-MM-DD e ponto decimal nos valores;
- tratar o beneficiário do boleto como emitente;

- procurar o pedido de compra nas observações;
- usar "null" para o que não existir, em vez de inventar;
- reduzir a confiança se algo estiver ilegível ou se os totais não fecharem.

A resposta. Com `client.messages.parse(..., output_format=ExtracaoLLM)`, a API obriga o modelo a responder no formato do esquema. O SDK já entrega um objeto Python validado. Não há "recortar texto" da resposta.

O escalonamento. Primeiro tenta o Haiku 4.5, mais barato. Se a confiança vier abaixo de 0,85, repete com o Sonnet 5.5. Se ainda vier baixa, a regra `confianca_extracao` manda para revisão.

A dupla checagem. Mesmo com confiança alta, os dados extraídos passam pelas mesmas regras de um XML: DV da chave, soma dos itens e match com o pedido. Se a IA errasse um número, uma dessas contas provavelmente não fecharia.

8.2 O agente investigador

O agente só é chamado quando as regras encontram ALERTA ou CRÍTICO.

O que ele recebe. O documento extraído, a lista de verificações e a decisão que as regras tomariam. Tudo vai em JSON.

O papel (prompt de sistema). "Você é o agente de Contas a Pagar de uma indústria". Suas tarefas: investigar com as ferramentas quando houver algo incomum, decidir chamando `registrar_decisao` uma única vez e escrever uma justificativa curta (2 a 4 frases) com ação sugerida. A política é explícita: crítico vira rejeitado, alerta vira no mínimo revisão e, na dúvida, revisão.

As ferramentas. As quatro são declaradas com `strict: true`, o que garante que os parâmetros venham no formato certo:

Ferramenta	Entrada	Devolve
<code>consultar_pedido</code>	número do pedido	Fornecedor e itens com quantidade e preço
<code>consultar_recebimento</code>	número do pedido	Quantidades que chegaram
<code>historico_fornecedor</code>	CNPJ	Cadastro e documentos já processados
<code>registrar_decisao</code>	status, justificativa, ação sugerida	Encerra a investigação

Um exemplo real do fluxo, para a nota 4 (luvas):

1. O agente lê: "faturado 500, recebido 400".
2. Chama `consultar_recebimento("PC-1004")` e confirma 400 luvas e 100 óculos.
3. Chama `registrar_decisao(REVISA0, "Faltaram 100 luvas...", "Pedir NF complementar ou devolução")`.
4. O código grava a decisão como `agente:claude-sonnet-5-5`.

Esse roteiro é exatamente o que o teste `test_loop_do_agente_com_cliente_simulado` reproduz.

Detalhes de implementação:

- **Loop manual:** até 6 voltas.
- **Escolha da ferramenta automática:** `tool_choice` fica no modo automático, porque os modelos atuais não aceitam forçar uma ferramenta. Se o agente responder sem chamar ferramenta, o código pede "Finalize chamando registrar_decisao".
- **Histórico completo:** a resposta inteira volta para o histórico, inclusive os blocos de "pensamento" do modelo.
- **Esforço médio:** `effort: "medium"` equilibra profundidade de raciocínio e custo.
- **Erros de API:** qualquer `anthropic.APIError` faz cair na decisão das regras.

8.3 Modo offline

Sem `ANTHROPIC_API_KEY`, nada quebra:

Etapa	Com IA	Sem IA
XML	Parser	Parser (igual)
PDF	Claude, confiança informada	Regex, confiança fixa de 0,6
Decisão de exceções	Agente + guard-rail	Só regras
Resultado da demonstração	5 aprovados, 2 revisões, 4 rejeitados (esperado)	4 aprovados, 3 revisões, 4 rejeitados (testado)

A diferença está só na DANFE 008, que sem IA vai para revisão por cautela.

8.4 Custos estimados da IA

Preços por milhão de tokens: Haiku 4.5 custa US\$ 1 de entrada e US\$ 5 de saída; Sonnet 5.5, US\$ 2 e US\$ 10.

Uso	Tokens por documento	Custo por documento
Ler um PDF (Haiku)	~3 mil de entrada, ~600 de saída	~US\$ 0,006
Rer ler um PDF (Sonnet)	~3 mil de entrada, ~1 mil de saída	~US\$ 0,016
Investigar uma exceção (Sonnet, 3 a 4 voltas)	~12 mil de entrada, ~2 mil de saída	~US\$ 0,044

Com 2.000 documentos por mês (30% só PDF, 25% de exceções), a estimativa é de cerca de US\$ 27 por mês.

8.5 Validação com IA real

O caminho com IA foi validado em 06/10/2026 com o modelo gratuito `nvidia/nemotron-3-super-120b-a12b:free`, via OpenRouter. Hoje não há Qwen gratuito com suporte a ferramentas lá.

Teste	Resultado
Demonstração completa (11 documentos)	5 aprovados, 2 em revisão, 4 rejeitados: o esperado com IA
DANFE em PDF	Lida pelo modelo (CNPJ, valor, vencimento, pedido, item) e aprovada
6 exceções	O agente consultou as ferramentas e escreveu justificativa e ação sugerida; o guard-rail manteve os 4 rejeitados
E-mail real (Gmail)	Anexos baixados, processados e auditados; e-mails sem anexo ignorados
Documento real recebido	Uma NFS-e de serviço: o PDF foi lido e rejeitado por prestador fora do cadastro; o XML municipal foi identificado como NFS-e, fora do escopo

Exemplo de justificativa do agente (nota 003): "Preço unitário do item OL-68 está R\$ 420,12, 8,0% acima do preço do pedido (R\$ 389,00), ultrapassando a tolerância de 2%. Ação sugerida: solicitar ao fornecedor uma justificativa para o aumento ou uma carta de correção."

A reserva também foi validada. Com o OpenRouter sem cota (HTTP 429), a cadeia passou para o Gemini e a demonstração terminou com o mesmo resultado, sem intervenção.

8.6 Usar um modelo gratuito (OpenRouter + Gemini)

O Claude é o padrão, mas a IA pode ser trocada por um modelo gratuito só mudando o `.env`:

```
AP_LLM_PROVEDOR=openai_compat
AP_LLM_BASE_URL=https://openrouter.ai/api/v1
AP_LLM_API_KEY=sk-or-...
AP_LLM_MODELO=nvidia/nemotron-3-super-120b-a12b:free

# reserva (Gemini gratuito, chave em aistudio.google.com/apikey)
AP_LLM_RESERVA_API_KEY=...
AP_LLM_RESERVA_MODELO=gemini-3.5-flash,gemini-3.5-flash-lite,gemini-3.7-flash
```

Aspecto	Claude	Modelo gratuito (OpenRouter)
Custo	~US\$ 27/mês no volume de produção estimado	Zero, com limite diário de chamadas
Leitura de PDF	Nativa, inclusive escaneado	Pelo texto extraído; não lê PDF escaneado
Garantia de formato	Schema obrigatório	Pedido ao modelo; validado pelo Pydantic, com nova tentativa
Privacidade	Contrato comercial de dados	No plano gratuito, o provedor pode usar os dados
Disponibilidade	Alta	Pode variar; falhas caem no modo offline

Para a demonstração com dados fictícios, o modelo gratuito basta. Para produção com notas de fornecedores, o recomendado é um provedor com contrato de dados. A arquitetura permite trocar sem mexer no código.

8.7 A cadeia de provedores e o disjuntor de cota

Modelos gratuitos têm cota diária: cerca de 50 chamadas no OpenRouter e cerca de 20 por modelo no Gemini. Para não depender de um só, o `llm.py` monta uma **cadeia** (`ProvedorEmCadeia`):

- Principal:** o provedor do `.env` (OpenRouter ou Claude).

- 2. **Reservas:** um provedor para cada modelo de `AP_LLM_RESERVA_MODELO` , na ordem da lista (Gemini Flash, Flash-Lite, ...).
- 3. **Último recurso:** se todos falharem, a extração usa regex e a decisão fica com as regras. Nenhum documento fica sem decisão.

Disjuntor. Quando um modelo responde "cota esgotada" (429 ou `RESOURCE_EXHAUSTED`), ele é marcado como `esgotado` e pulado no resto do lote. Sem isso, cada um dos 11 documentos perderia segundos tentando um modelo que já se sabe indisponível. Outros erros (503 de sobrecarga, JSON inválido) não desligam o modelo: só passam a vez ao próximo daquela chamada.

Uma cadeia por processo. `provedor_do_processo()` guarda a cadeia, para que o disjuntor valha em todo o lote, inclusive entre a leitura do PDF e a investigação do agente.

Rastreabilidade. O campo `metodo_extracao` e o "decidido por" gravam qual modelo de fato respondeu (ex.: `agente:gemini-3.5-flash`). O topo do painel mostra a cadeia configurada, por exemplo: `nemotron... via openrouter -> reserva: gemini-3.5-flash, ....`

Renovação das cotas.

- OpenRouter: à meia-noite UTC (21h de Brasília).
- Gemini: diariamente, no horário do Pacífico (às 4h ou 5h de Brasília).

9. As interfaces: painel, terminal e MCP

As três interfaces chamam as mesmas funções (`processar_pasta` , `store.listar` , `store.atualizar_status`). Por isso mostram sempre os mesmos dados.

9.1 Painel Streamlit (`dashboard.py`)

Como funciona o Streamlit: o script Python roda de cima a baixo a cada clique, e cada comando (`st.metric` , `st.button` , `st.tabs`) desenha um elemento na página. Não há HTML nem JavaScript escritos à mão.

Parte do painel	Comando Streamlit	O que chama
Botão "Processar novos"	<code>st.button</code>	<code>processar_pasta</code> só com arquivos ainda não lançados
Envio de arquivo	<code>st.file_uploader</code>	Salva na inbox e chama <code>processar_documento</code>
Indicadores	<code>st.metric</code>	Cálculos sobre a tabela <code>documentos</code> (pandas)

Parte do painel	Comando Streamlit	O que chama
Fila de revisão	<code>st.expander</code> + botões	<code>store.atualizar_status</code> com <code>humano:<nome></code>
Vencimentos	<code>st.slider</code> + <code>st.dataframe</code>	Filtra aprovados até a data-limite
Detalhes	<code>st.json</code>	Mostra o <code>dados_json</code> do documento
Auditoria	<code>st.dataframe</code>	Lê a tabela <code>auditoria</code>

O painel informa no topo a IA em uso (a cadeia de provedores, ou "offline"), a base de dados aberta e que os dados são fictícios.

9.2 Terminal (CLI)

Pensada para automação: um agendador do Windows pode rodar `python -m ap_agent processar` a cada hora, sem ninguém abrir o painel. Os comandos estão na seção 2.10.

9.3 Servidor MCP (`mcp_server.py`)

Como é construído. Com o SDK oficial `mcp` (versão 2), cria-se um `MCPServer("contas-a-pagar")`. Cada função Python marcada com `@mcp.tool()` vira uma ferramenta. O SDK lê o nome, os parâmetros com seus tipos e o texto de documentação da função, e monta a descrição que o Claude vê. O texto de documentação é o que ensina o Claude a usar a ferramenta, por isso é escrito com cuidado.

As 8 ferramentas e o recurso:

Ferramenta	Parâmetros	O que faz
<code>buscar_emails</code>	<code>processar</code> (padrão: sim)	Lê a caixa de e-mail, baixa os anexos XML/PDF e processa cada documento
<code>processar_documentos</code>	<code>arquivo</code> (opcional)	Processa os arquivos novos da inbox, ou um específico
<code>listar_pendencias</code>	nenhum	Fila de revisão com motivos
<code>listar_documentos</code>	<code>status</code> (opcional)	Lançamentos filtrados por status
<code>detalhar_documento</code>	<code>documento_id</code>	Dados extraídos + todas as verificações

Ferramenta	Parâmetros	O que faz
<code>decidir_pendencia</code>	<code>documento_id</code> , <code>aprovar</code> , <code>justificativa</code> , <code>analista</code>	Aprova ou rejeita um item em REVISÃO, com auditoria
<code>relatorio_vencimentos</code>	<code>dias</code> (padrão 7)	Títulos aprovados no período, com total
<code>consultar_fornecedor</code>	<code>cnpj</code>	Cadastro + histórico
<code>recurso</code> <code>contas-a-pagar://kpis</code>	—	Volume por status e % decidido sem humano

Uma trava importante. `decidir_pendencia` só aceita documentos em REVISÃO. Um documento REJEITADO por regra crítica, como o golpe do boleto, não pode ser aprovado pelo chat, nem por engano.

Os dois transportes:

- **stdio**, o padrão: registrado com `claude mcp add contas-a-pagar -- python "<caminho>/mcp_server.py"`. O Claude Code abre o programa e conversa com ele pela entrada e saída padrão. Nada fica exposto na rede.
- **HTTP**: `python mcp_server.py --http` sobe em `http://127.0.0.1:8000/mcp`. Serve para clientes de rede, como o nó *MCP Client* do n8n, que liga este desafio ao Desafio 2.

O que acontece numa conversa. Quando você pergunta "o que está pendente?":

1. O Claude Code envia ao modelo sua pergunta e a lista de ferramentas disponíveis.
2. O modelo decide chamar `listar_pendencias`.
3. O Claude Code repassa a chamada ao servidor, que consulta o SQLite e devolve um JSON.
4. O modelo lê o JSON e responde em português, por exemplo: "Há 3 pendências: a DANFE de frete (confiança baixa), o óleo hidráulico (preço 8% acima)..."

Por isso o agente fica "conversacional" sem precisar de uma tela nova.

10. Testes, qualidade e tecnologias

A suíte tem 18 testes automatizados, e todos passam em cerca de 3 segundos (`python -m pytest -q` ; última execução: 06/10/2026, 18 passed). Os testes nunca chamam uma IA de verdade, mesmo com o `.env` preenchido: uma *fixture* automática zera as chaves. Além deles, o servidor MCP, o painel, a IA real e o e-mail real foram testados de ponta a ponta.

10.1 Os testes automatizados (`tests/test_agente.py`)

Teste	O que prova
<code>test_cnpj</code>	Aceita CNPJ válido; recusa DV errado e números repetidos
<code>test_chave_acesso</code>	Gera chave válida e detecta um dígito alterado
<code>test_linha_digitavel_roundtrip_e_adulteracao</code>	Gera, decodifica (valor e vencimento corretos) e detecta valor adulterado
<code>test_parser_xml</code>	Lê a NF-e 002: 2 itens, pedido PC-1002, R\$ 8.370,00, chave válida
<code>test_regras_decisao</code>	OK → aprovado; alerta → revisão; crítico → rejeitado
<code>test_guardrail_llm_nao_pode_ser_mais_permissivo</code>	Agente simulado diz "aprovar" num caso crítico e o resultado é REJEITADO
<code>test_vencimento_proximo_nao_bloqueia</code>	Vence amanhã → OK com PRIORIDADE
<code>test_ponta_a_ponta_offline</code>	Os 11 documentos fictícios saem todos com o status esperado
<code>test_reprocessar_e_bloqueado_como_duplicado</code>	Processar a pasta duas vezes rejeita os repetidos
<code>test_loop_do_agente_com_cliente_simulado</code>	O loop agente → ferramenta → resultado → decisão funciona sem chamar a API

Testes acrescentados na segunda versão:

Teste	O que prova
<code>test_ingestao_email_salva_so_xml_e_pdf_e_marcado_lido</code>	Com um servidor IMAP simulado: salva XML e PDF, ignora <code>.exe</code> , impede nome com <code>../</code> , marca os e-mails como lidos e audita
<code>test_pedido_inferido_quando_nota_nao_informada</code>	Nota sem pedido acha o PC-1002 pelo fornecedor e itens; caso ambíguo continua em alerta

Teste	O que prova
<code>test_vencimento_pelo_prazo_padrao_do_fornecedor</code>	Sem duplicata, vencimento = emissão + 30 dias do cadastro
<code>test_pastas_configuraveis_por_ambiente</code>	<code>AP_DATA_DIR</code> e <code>AP_INBOX_DIR</code> são respeitados
<code>test_provedor_openai_extrai_pdf_com_fallback_de_formato</code>	Modelo simulado recusa o JSON Schema; o sistema tenta o modo JSON simples e lê a DANFE
<code>test_provedor_openai_loop_do_agente</code>	Loop do agente no formato OpenAI: consulta o recebimento e registra a decisão

Testes da cadeia de provedores (terceira versão):

Teste	O que prova
<code>test_cadeia_de_provedores_usa_a_reserva_quando_o_principal_falha</code>	Principal simulado falha; a reserva responde, e o método registra o modelo da reserva
<code>test_disjuntor pula_modelo_com_cota_esgotada</code>	Após um 429, o modelo esgotado não é chamado de novo no lote; o próximo da lista assume

Isolamento. Uma *fixture* (preparação automática do pytest) gera os dados fictícios numa pasta temporária e aponta a configuração para ela. Os testes nunca tocam nos dados reais da demonstração.

10.2 Outras verificações feitas

- **Servidor MCP:** um cliente MCP de verdade, em Python, listou as 8 ferramentas e chamou `listar_pendencias`, `relatorio_vencimentos` e `decidir_pendencia`, além de ler o recurso de KPIs.
- **Painel:** o modo de teste do Streamlit (`AppTest`) carregou o painel, clicou em "Processar" e conferiu 11 documentos, 4 aprovados, 3 em revisão e R\$ 41.140,00 bloqueados.
- **Terminal:** `processar --offline`, `vencimentos` e `exportar` rodaram de ponta a ponta.
- **Atalho `demo.ps1`** : sobe o painel e abre o navegador.

10.3 Tecnologias usadas

Tecnologia	Versão	Papel
Python	3.12	Linguagem de toda a aplicação
anthropic (SDK)	1.x	Chamadas à Claude API
openai (SDK)	3.x	Cliente para APIs compatíveis: OpenRouter e Gemini
Claude Haiku 4.5 e Sonnet 5.5	—	Leitura de PDF e agente
mcp (SDK)	2.3	Servidor MCP
Pydantic	2.x	Moldes e validação de dados
SQLite	nativo do Python	Banco de dados
Streamlit	1.58	Painel web
pandas e openpyxl	—	Tabelas e exportação para Excel
pypdf	—	Texto de PDF no modo offline
reportlab	—	Geração dos PDFs fictícios
python-dotenv	—	Leitura do <code>.env</code>
pytest	8.4	Testes
Claude Code	—	Ferramenta de apoio usada para desenvolver tudo
Git	—	Controle de versões

11. Da demonstração à produção

O plano completo, em `docs/plano-implantacao.md`, leva o agente à operação real em cerca de 10 semanas. A economia líquida estimada é de R\$ 7 mil por mês, com retorno em 1 a 3 meses. Os números partem de premissas a validar: 2.000 documentos por mês, 3 analistas, R\$ 45 por hora e 6 minutos por documento hoje.

Do protótipo à produção: 10 semanas, 4 portões



plano de implantação · 4 fases, 4 portões

Cada fase só começa quando o portão anterior é cumprido. O portão mais exigente é o do piloto: zero documentos aprovados indevidamente.

11.1 O que muda do protótipo para a produção

Hoje (demonstração)	Produção
Pasta inbox	E-mail corporativo capturado por n8n ou Microsoft Graph
Cadastros em CSV	API ou visão somente leitura do ERP
Sem consulta à SEFAZ	Consulta da situação da NF-e (autorizada ou cancelada), com certificado A1
SQLite	PostgreSQL
Execução manual	Fila com novas tentativas automáticas, em container
Painel aberto	Painel com login corporativo (SSO)
Sem escrita no ERP	Pré-lançamento no ERP só dos aprovados; o pagamento continua com alçada humana

11.2 Custos e retorno

Item	Estimativa
Claude API	~US\$ 27/mês (~R\$ 150)
Infraestrutura, certificado e monitoramento	R\$ 350 a 650/mês

Item	Estimativa
Total recorrente	~R\$ 500 a 800/mês
Trabalho manual hoje	200 h/mês = R\$ 9.000/mês
Trabalho manual com o agente	25 h/mês = R\$ 1.125/mês
Economia líquida	~R\$ 7.000/mês e ~175 h liberadas

Ganhos fora da conta, e geralmente maiores: duplicidades evitadas, golpes barrados, multas evitadas e descontos por pagamento antecipado.

11.3 Principais riscos e mitigações

Risco	Mitigação
A IA lê um campo errado no PDF	Validações independentes, reextração, guard-rail e preferência pelo XML
Aprovação indevida	Shadow mode com meta zero; pagamento final continua humano
Atraso na integração com o ERP	Começar pela leitura; plano B com exportação agendada em CSV
LGPD e sigilo fiscal	Só PDFs sem XML vão à IA; retenção mínima; contrato de dados
API indisponível	Fila com novas tentativas e queda automática para as regras
Resistência do time	Analistas envolvidos desde a descoberta; o agente acaba com a digitação, não com o cargo

O plano completo tem 9 riscos, além de KPIs com metas, matriz de responsabilidades e regras de governança para mudar regras ou prompts.

11.4 Limitações atuais

- Modelos gratuitos têm cota diária (cerca de 50 + 20 por modelo Gemini) e às vezes ficam sobrecarregados. A cadeia de reservas cobre a maior parte disso; se todas falharem, o documento é decidido pelas regras, e um PDF vai para revisão.
- A leitura de e-mail usa IMAP com senha de app. Em ambiente corporativo (Microsoft 365), o recomendado é OAuth via Microsoft Graph.

- Não cobre NFS-e (notas de serviço), CT-e nem impostos retidos. O XML de NFS-e é identificado e registrado na auditoria.
- Não recalcula impostos: só confere somas.
- SQLite não é indicado para vários usuários escrevendo ao mesmo tempo.

11.5 Próximos passos

1. Rodar em shadow mode com notas reais da empresa, comparando com a decisão dos analistas (fase 2 do plano).
2. Trocar o modelo gratuito por um provedor com contrato de dados (Claude) antes de processar notas de fornecedores.
3. Ler e-mail corporativo por OAuth (Microsoft Graph) em vez de senha de app.
4. Conectar o servidor MCP ao n8n para gerar conteúdo a partir de dados financeiros, unindo os dois desafios.
5. Fase 2 do produto: NFS-e, conciliação bancária e previsão de caixa.

Já concluídos nesta versão: validação com IA real (OpenRouter e Gemini), leitura de e-mail real e teste com documentos reais.